UNIVERSIDAD CATÓLICA DE CUENCA

FACULTAD DE INFORMÁTICA, CIENCIAS DE LA COMPUTACIÓN E INNOVACIÓN TECNOLÓGICA

CARRERA DE ROBÓTICA E INTELIGENCIA ARTIFICIAL

ASIGNATURA: MICROCONTROLADORES

TEMA: SISTEMA INTELIGENTE DE CONTROL DE TRÁFICO CON ESP32

NOMBRE DEL ESTUDIANTE:

Mateo Roca

DOCENTE: ING. PABLO BUESTÁN ANDRADE, PhD

FECHA: 1/05/2026

INTRODUCCION

La programación basada en interrupciones representa un paradigma avanzado en el desarrollo de software para microcontroladores. A diferencia del pooling tradicional, las interrupciones de hardware permiten que el ESP32 responda a eventos temporales con alta precisión. En esta práctica, se explora la implementación de una máquina de estados que gestiona la modulación por ancho de pulso (PWM) y secuencias digitales, garantizando un entorno de ejecución libre de funciones bloqueantes.

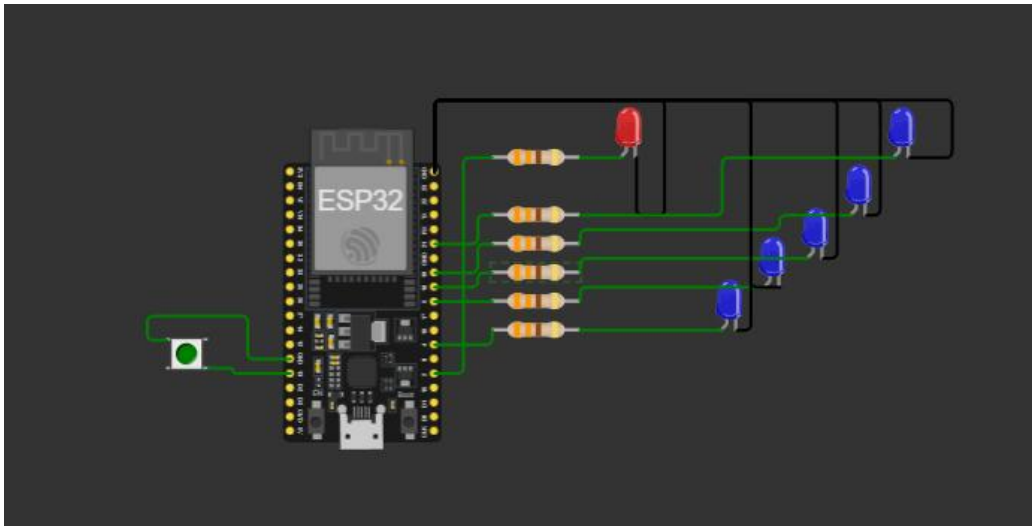
OBJETIVO GENERAL

Implementar un sistema de control de efectos lumínicos mediante el microcontrolador ESP32, utilizando temporizadores de hardware y modulación de ancho de pulso para la creación de comportamientos dinámicos sin el uso de retardos de software.

OBJETIVOS ESPECÍFICOS

- Configurar interrupciones de timer para la creación de una base de tiempo.
- Implementar el control de intensidad LED mediante analogWrite() sincronizado con la ISR.
- Desarrollar un secuenciador lógico de 5 niveles con tiempos de conmutación definidos por software no bloqueante.

DISEÑO ESQUEMATICO



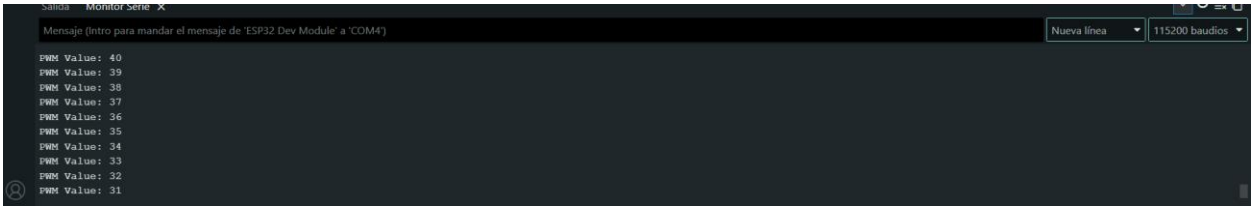
CÓDIGO

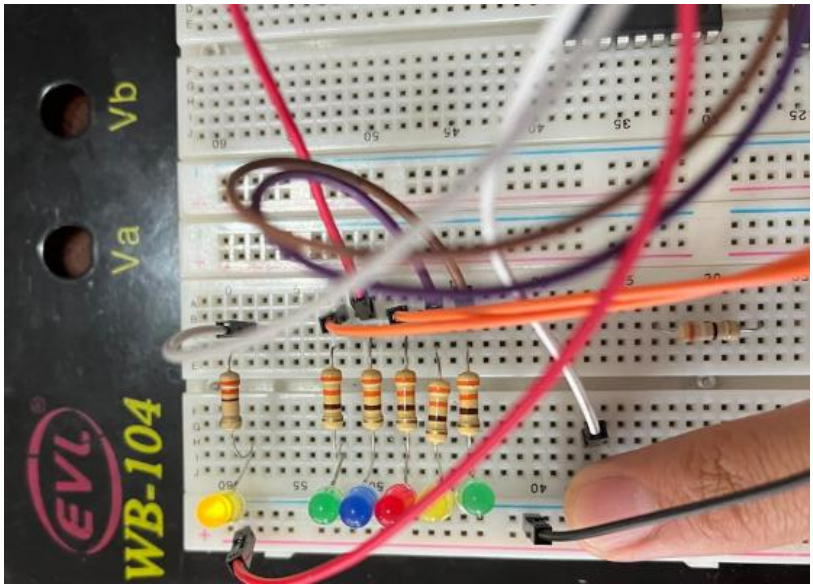
```
 * ACTIVIDAD PRÁCTICA: Máquina de Estados con ISR (ESP32)
 * Estudiante: [MATEO ROCA]
 * Sin uso de delay() ni millis().
 */

// --- Pines ---
const int PIN_PWM = 2;
const int LEDS_RULETA[] = {4, 5, 18, 19, 21};
const int PIN_BOTON = 13;

// --- Variables de Estado (Volatile para ISR) ---
enum Estados { BREATHING, RULETA };
volatile Estados estadoActual = BREATHING;
volatile bool huboCambioBoton = false;
volatile int contadorTicks = 0;

// Control de efectos
int brillo = 0;
int direccion = 1;
int indiceRuleta = 0;
int subContadorRuleta = 0;

hw_timer_t * timer = NULL;

// --- ISR: Botón ---
void IRAM_ATTR manejarBotonISR() {
  huboCambioBoton = true;
}

// --- ISR: Timer (Base de tiempo 10ms) ---
void IRAM_ATTR onTimer() {
  contadorTicks++;
}

void setup() {
  Serial.begin(115200);

  pinMode(PIN_PWM, OUTPUT);
  pinMode(PIN_BOTON, INPUT_PULLUP);
  for (int i = 0; i < 5; i++) pinMode(LEDS_RULETA[i], OUTPUT);

  // Interrupción externa para el botón
  attachInterrupt(digitalPinToInterrupt(PIN_BOTON), manejarBotonISR, FALLING);

  // Configuración del Timer de Hardware a 10ms
```

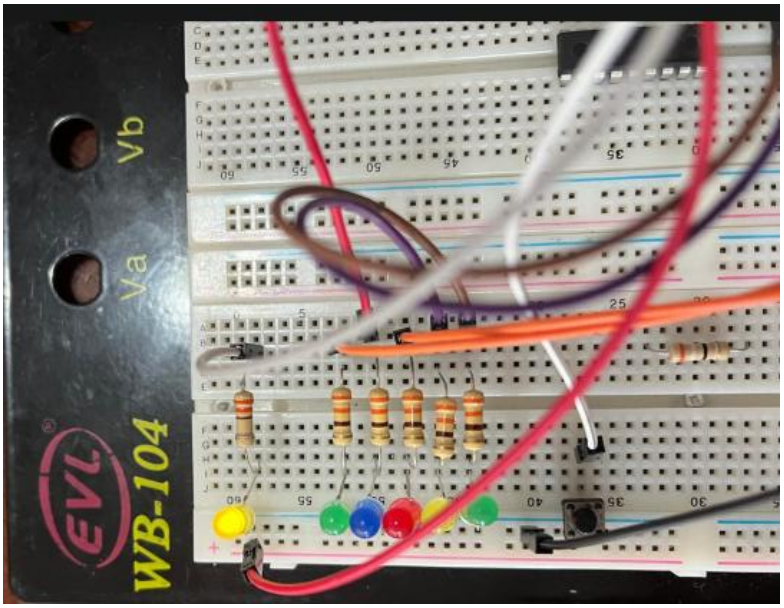
```
  timer = timerBegin(1000000);
  timerAttachInterrupt(timer, &onTimer);
  timerAlarm(timer, 10000, true, 0);
}

void loop() {
  // Cambio de estado detectado por ISR del botón
  if (huboCambioBoton) {
    estadoActual = (estadoActual == BREATHING) ? RULETA : BREATHING;
    limpiarSalidas();
    Serial.println("\n>>> CAMBIO DE ESTADO <<<");
    huboCambioBoton = false;
  }

  // Ejecución sincronizada por el Timer (cada 10ms)
  if (contadorTicks > 0) {
    if (estadoActual == BREATHING) {
      ejecutarBreathing();
    } else {
      ejecutarRuleta();
    }
    contadorTicks = 0;
  }
}

// MODO 1: Respiración PWM
void ejecutarBreathing() {
  analogWrite(PIN_PWM, brillo);
  Serial.print("PWM Value: ");
  Serial.println(brillo);

  brillo += direccion;
  if (brillo <= 0 || brillo >= 255) direccion *= -1;
}

// MODO 2: Ruleta (0.5s)
void ejecutarRuleta() {
  subContadorRuleta++;

  // 50 ticks de 10ms = 500ms (0.5s)
  if (subContadorRuleta >= 50) {
    for (int i = 0; i < 5; i++) digitalWrite(LEDS_RULETA[i], LOW);
    digitalWrite(LEDS_RULETA[indiceRuleta], HIGH);

    indiceRuleta++;
```

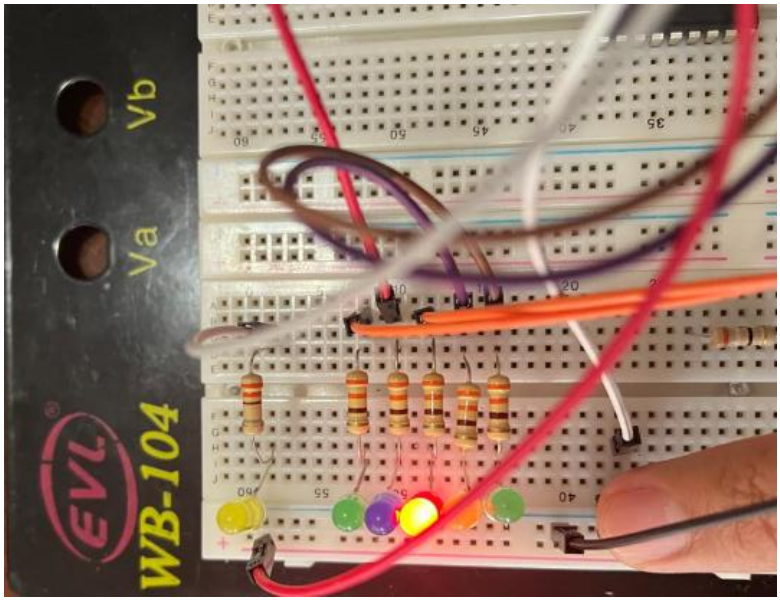
```
    if (indiceRuleta >= 5) indiceRuleta = 0;

    subContadorRuleta = 0;
  }
}

// Limpieza al cambiar de modo
void limpiarSalidas() {
  analogWrite(PIN_PWM, 0);
  for (int i = 0; i < 5; i++) digitalWrite(LEDS_RULETA[i], LOW);
  brillo = 0;
  indiceRuleta = 0;
  subContadorRuleta = 0;
}
```

FOTOGRAFIAS






Vb
Va
EVL
WB-104


Vb
Va
EVL
WB-104